

Ejemplos

Raul Gomez

2026-02-12

Protocolo de Encriptación RSA

En este notebook, mostraremos el proceso de encriptación RSA. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import (  
    rsa_key_generation,  
    rsa_encryption,  
    rsa_decryption,  
    encode,  
    decode  
)
```

Una vez que ya hemos cargado las librerías necesarias, empezamos por generar nuestras llaves *pública* y *privada*

```
(public_key, private_key) = rsa_key_generation(1000)
```

En este caso la llave privada consiste de dos partes. Es muy importante que esta información no se divulgue.

```
private_key
```

```
8958109057962614086000396135659468835564662042208417115926634094925555611654651935511232040451
```

La llave pública, por otro lado, se debe de compartir con las partes interesadas, a fin de que puedan encriptar sus mensajes. En este caso la llave pública a compartir es:

```
public_key
```

```
(401490110107356274942810187706574137513837813051814046777839704122883030377683654863202386899364503648415831770100740396872718570092520396472800543551995225666429702064760759776536263986)
```

Para demostrar como funciona este proceso, procedemos ahora a generar un mensaje que deseamos encriptar. Empezaremos con el simple 'Hello World!'

```
plain_text = 'Hello World!'
```

Antes de poder encriptar este mensaje, necesitamos codificarlo en forma numérica:

```
plain_message = encode(plain_text)
plain_message
```

```
10334410032597741434076685640
```

Utilizando la llave pública, podemos ahora generar el *mensaje encriptado*

```
encrypted_message = rsa_encryption(public_key, plain_message)
encrypted_message
```

```
2109491071755564097809536140486428561008869468135290700746036442012968985345898073041612142387
```

Si este mensaje fuera interceptado por un adversario (digamos Eva) este sería el mensaje de texto que obtendría:

```
encrypted_text = decode(encrypted_message)
encrypted_text
```

```
"\x15°\x1fà:ûN\x88\tÉü\xad\x97ø±\x82E÷\x0e87\x82>\x93\x95.k\x92ëü\x03XÓ+\x89ý·\x87©\x81'c,\x83
```

Por otro lado, Alicia al momento de recibir este mensaje podría desencriptarlo primero utilizando su llave pública:

```
decrypted_message = rsa_decryption(public_key,private_key, encrypted_message)
decrypted_message
```

```
10334410032597741434076685640
```

Para poder leer este mensaje basta con decodificarlo a mensaje de texto:

```
decrypted_text = decode(decrypted_message)
decrypted_text
```

```
'Hello World!'
```

Protocolo Diffie-Hellman para la generación de una llave compartida

En este notebook, mostraremos el proceso de generar una llave compartida utilizando el protocolo Diffie-Hellman. Empezaremos por cargar las funciones que necesitamos:

```
from cryptocalc import (
    RFC_3526_SAFE_PRIME,
    RFC_3526_SOPHIE_PRIME,
    generate_private_key,
    diffie_hellman_public_key_generator,
    diffie_hellman_shared_key_generator
)
```

Para este ejemplo, utilizaremos uno de los primos seguros listados en la especificación [RFC 3526](#) y generaremos las llaves secretas de Alicia y Beto de manera aleatoria:

```
p = RFC_3526_SAFE_PRIME
q = RFC_3526_SOPHIE_PRIME
g = 2
a = generate_private_key(q)
b = generate_private_key(q)

print(f"Primo Seguro (p): {p}\n")
print(f"Primo de Sophie Germain (q): {q}\n")
print(f"Generador (g): {g}\n")
print(f"Llave privada de Alicia (a): {a}\n")
print(f"Llave privada de Beto (b): {b}")
```

Primo Seguro (p): 1552518092300708935130918131258481755631334049434514313202351194902966239949

Primo de Sophie Germain (q): 77625904615035446756545906562924087781566702471725715660117559745

Generador (g): 2

Llave privada de Alicia (a): 70008936640472710207296716780966626838850707592089826082242902515

Llave privada de Beto (b): 1732076688990100277415344512343507890970098608882974798009337038181

Generamos ahora las llaves públicas de Alicia y Beto:

```
A = diffie_hellman_public_key_generator(p, g, a)
B = diffie_hellman_public_key_generator(p, g, b)

print(f"Llave pública de Alicia (A): {A}\n")
print(f"Llave pública de Beto (B): {B}")
```

Llave pública de Alicia (A): 10260866127083692818590708430452590252569790636817929402968588958

Llave pública de Beto (B): 7425899615685845332016880039184319405695763908219954979337982139526

Finalmente, generamos las llaves compartidas

```
s_a = diffie_hellman_shared_key_generator(p, B, a)
s_b = diffie_hellman_shared_key_generator(p, A, b)

print(f"Llave compartida generada por Alicia (s_a): {s_a}\n")
print(f"Llave compartida generada por Beto (s_b): {s_b}\n")
```

Llave compartida generada por Alicia (s_a): 45091057358503606771376186832899555897198861394791

Llave compartida generada por Beto (s_b): 4509105735850360677137618683289955589719886139479134

Como podemos observar, ambas llaves son iguales, lo cual ilustra la corrección del protocolo.